

1. Project Identity

Name

Prism IQ

Tagline

Transform Raw Data into Actionable Intelligence.

Project Type

AI-Powered Data Analytics Platform

Objective

Prism IQ enables users to import structured data from multiple sources, automatically profile, clean, analyze, visualize, generate predictive models, and produce business-ready reports through a guided analytics pipeline.

Target Users

- Students
 - Data Analysts
 - Business Analysts
 - Small Businesses
 - Researchers
 - Educators
-

2. Product Philosophy

Prism IQ is **not** a dashboard builder.

It is **not** another Power BI clone.

It is an **AI-assisted analytics workspace**.

The product should guide users from **raw data** → **business decision** with minimal manual effort.

3. Core Principles

- Pipeline-first architecture.
 - Original datasets are immutable.
 - Modular services.
 - Clear UX.
 - REST-first APIs.
 - Metadata in PostgreSQL.
 - Files stored separately.
 - Reusable UI components.
 - No duplicate business logic.
-

4. Tech Stack

Frontend

- React
 - TypeScript
 - Vite
 - Tailwind CSS
 - shadcn/ui
 - TanStack Query
 - Plotly
 - Recharts
 - React Hook Form
 - Zod
 - Framer Motion
 - Lucide React
-

Backend

- FastAPI
- SQLAlchemy 2.0
- Alembic
- PostgreSQL
- Pydantic
- Pandas

- NumPy
 - Scikit-learn
 - BeautifulSoup4
 - HTTPX
 - ReportLab
-

Development

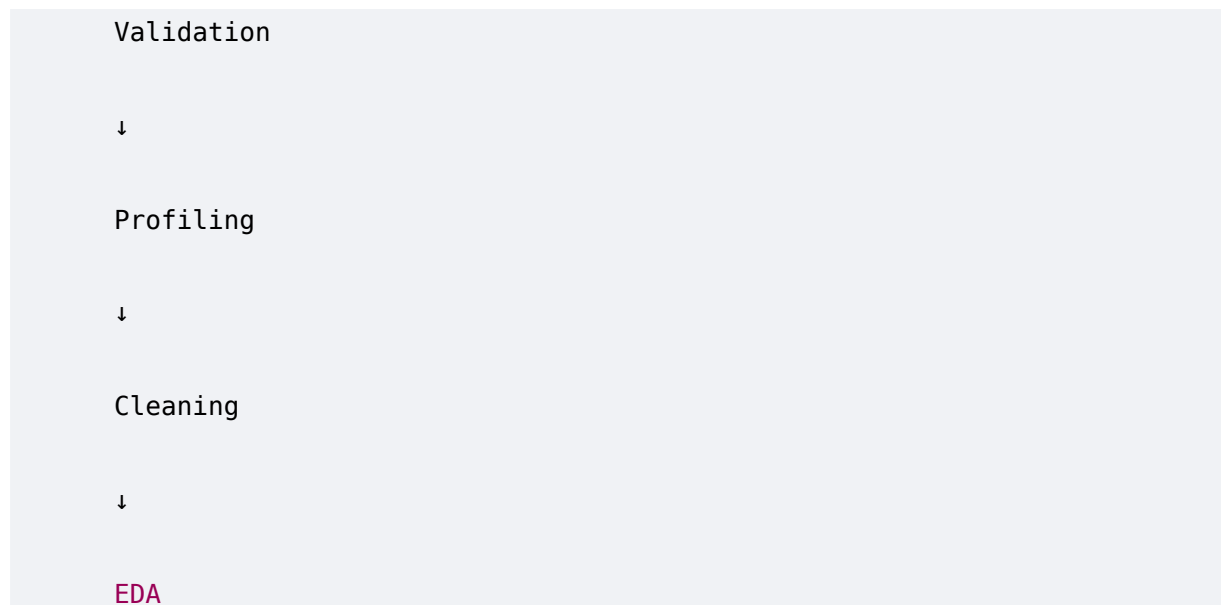
- Git
 - GitHub
 - Docker (future)
 - Uvicorn
-

5. Folder Structure

```
prism-iq/  
  
docs/  
  
frontend/  
  
backend/  
  
uploads/  
  raw/  
  cleaned/  
  
reports/  
  
models/  
  
tests/
```

6. Application Pipeline

```
Import  
  
↓
```



Every dataset follows this lifecycle.

7. Database Philosophy

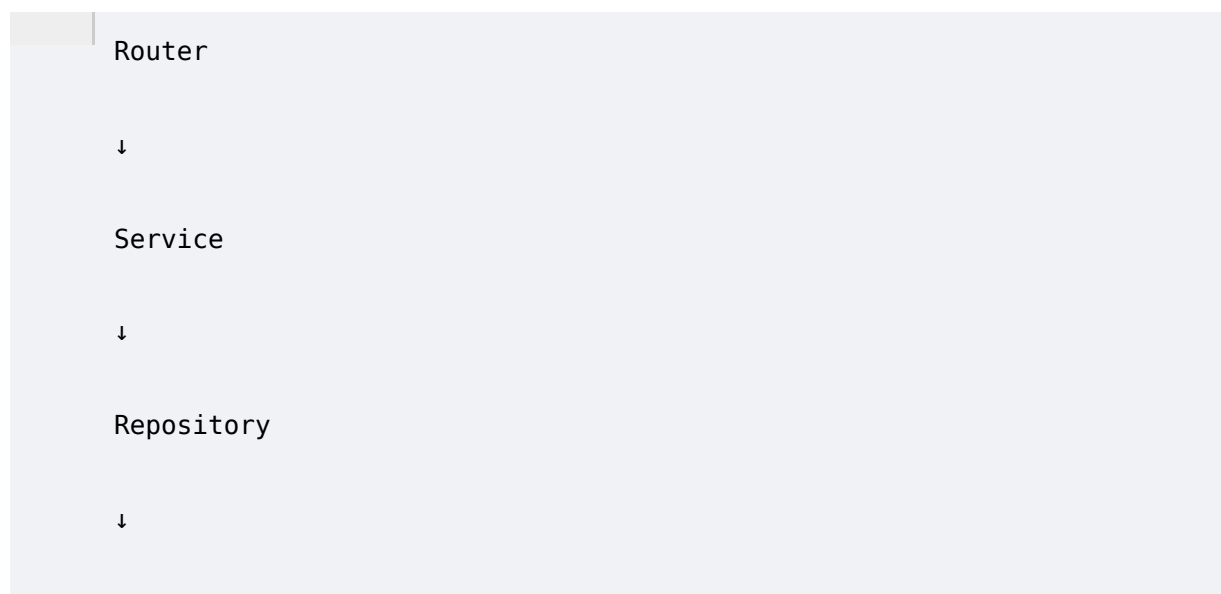
PostgreSQL stores:

- Metadata
- Reports metadata
- ML metadata
- Analysis metadata

Never store raw datasets inside PostgreSQL.

Large files remain on disk.

8. Backend Architecture



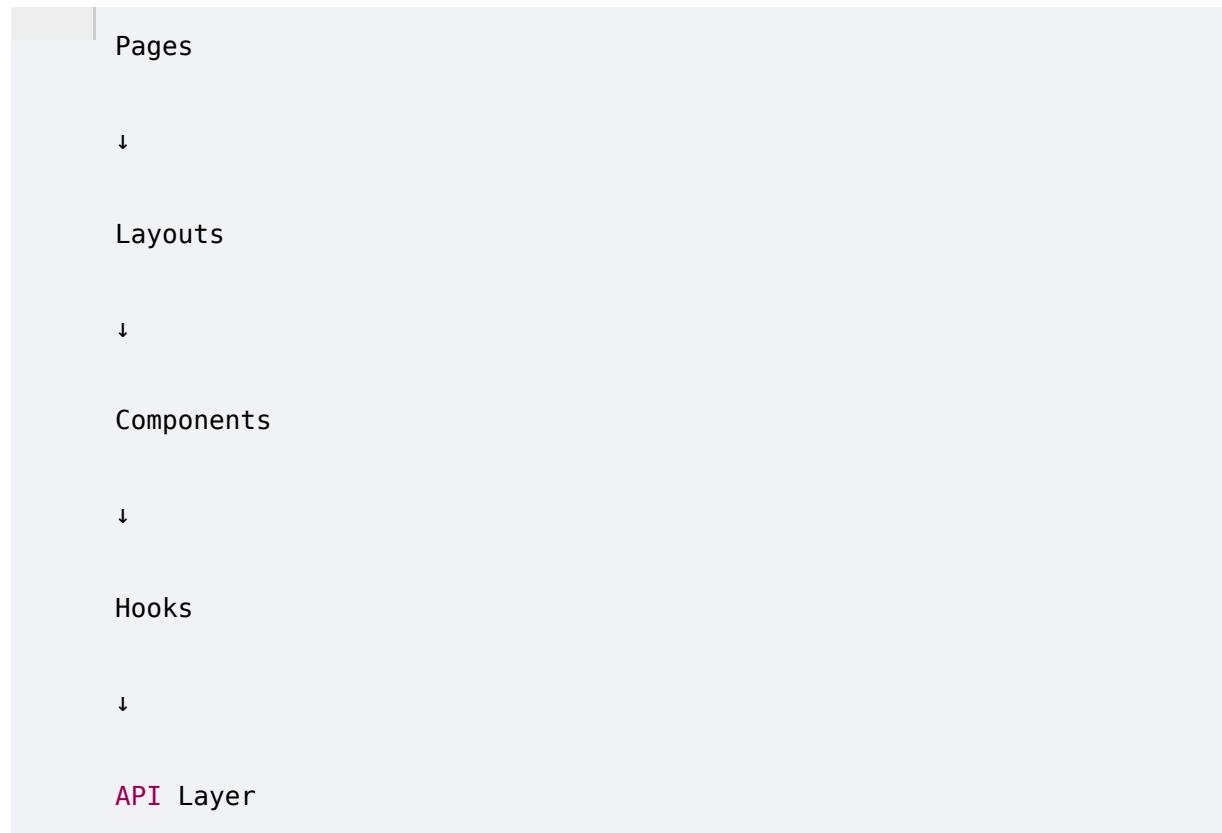
Database

Routes never contain business logic.

Repositories never call external services.

Services orchestrate everything.

9. Frontend Architecture



Business logic belongs to backend.

Frontend focuses on rendering and interaction.

10. Design Language

Theme:

Premium AI SaaS

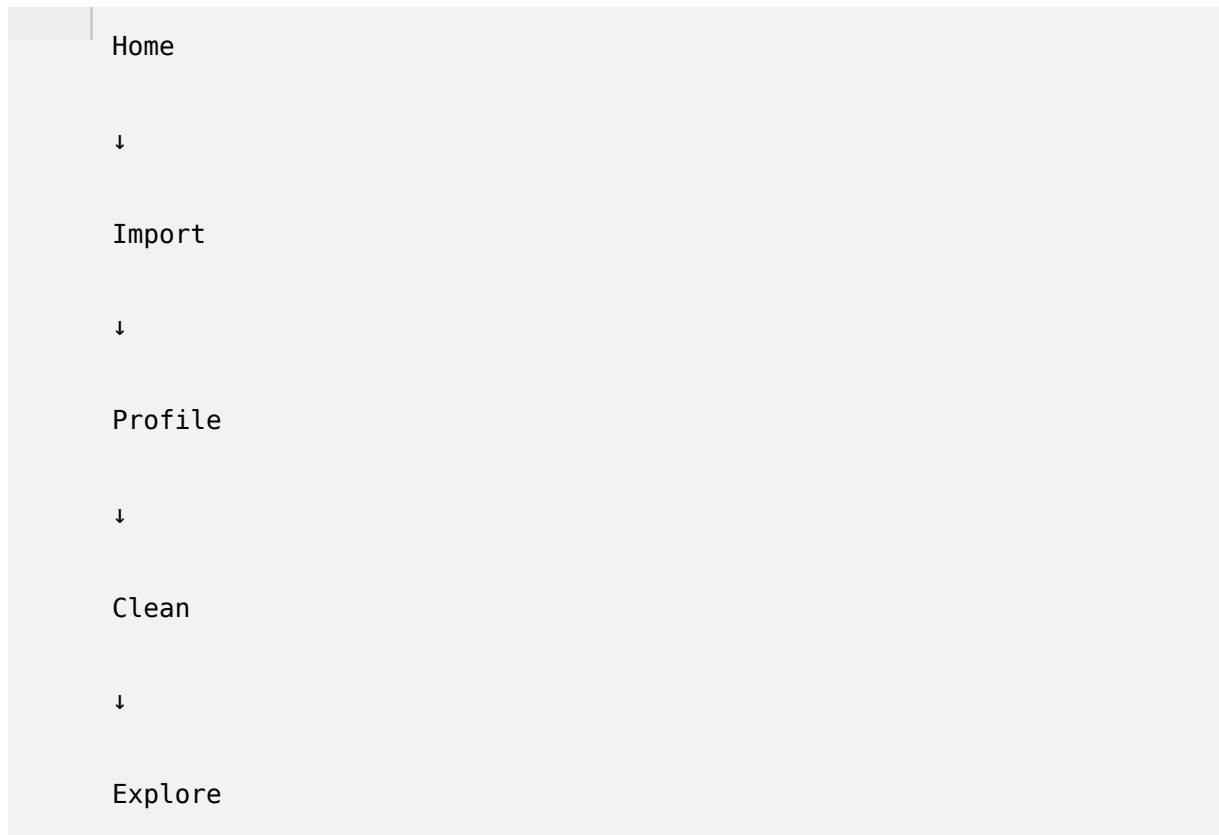
Style:

Minimal

Elegant

Workspace-based

11. Navigation



This mirrors the analytics pipeline.

12. Color Tokens

Brand Colors:

- Prism Blue
- Prism Violet
- Cyan Accent

Semantic:

- Success
- Warning
- Error
- Info

Background:

Layered dark surfaces with subtle elevation.

13. Typography

Heading:

Space Grotesk

Body:

Inter

Code:

JetBrains Mono

14. API Standards

- RESTful.
 - JSON only.
 - Standard response wrapper.
 - Predictable routes.
 - Proper HTTP status codes.
 - Validation before execution.
-

15. Coding Standards

Backend

- Async wherever appropriate.
 - Type hints required.
 - Dependency injection for shared services.
 - Separate schemas from ORM models.
 - One responsibility per module.
-

Frontend

- Functional components.
 - TypeScript strict mode.
 - Reusable components.
 - No duplicated UI logic.
 - Custom hooks for shared state.
-

16. Naming Conventions

Python:

```
snake_case
```

React Components:

```
PascalCase
```

Variables:

```
camelCase
```

Constants:

```
UPPER_CASE
```

API Routes:

```
kebab-case
```

17. Error Handling

Every endpoint returns:

```
{
  "success": true,
  "message": "",
  "data": {},
  "errors": []
}
```

Never expose stack traces to the client.

18. Logging

Log:

- Uploads
- Cleaning
- Profiling
- ML Training
- Report Generation
- API Errors

Future:

Structured JSON logs.

19. Machine Learning Workflow

1. User selects target.
 2. Detect regression/classification.
 3. Preprocess data.
 4. Train multiple models.
 5. Compare metrics.
 6. Save best model.
 7. Generate predictions.
-

20. Report Philosophy

Reports should be concise and decision-oriented.

Include:

- Executive Summary
- Dataset Overview
- Cleaning Summary
- Key Insights
- Model Results
- Recommendations

Avoid dumping raw statistics.

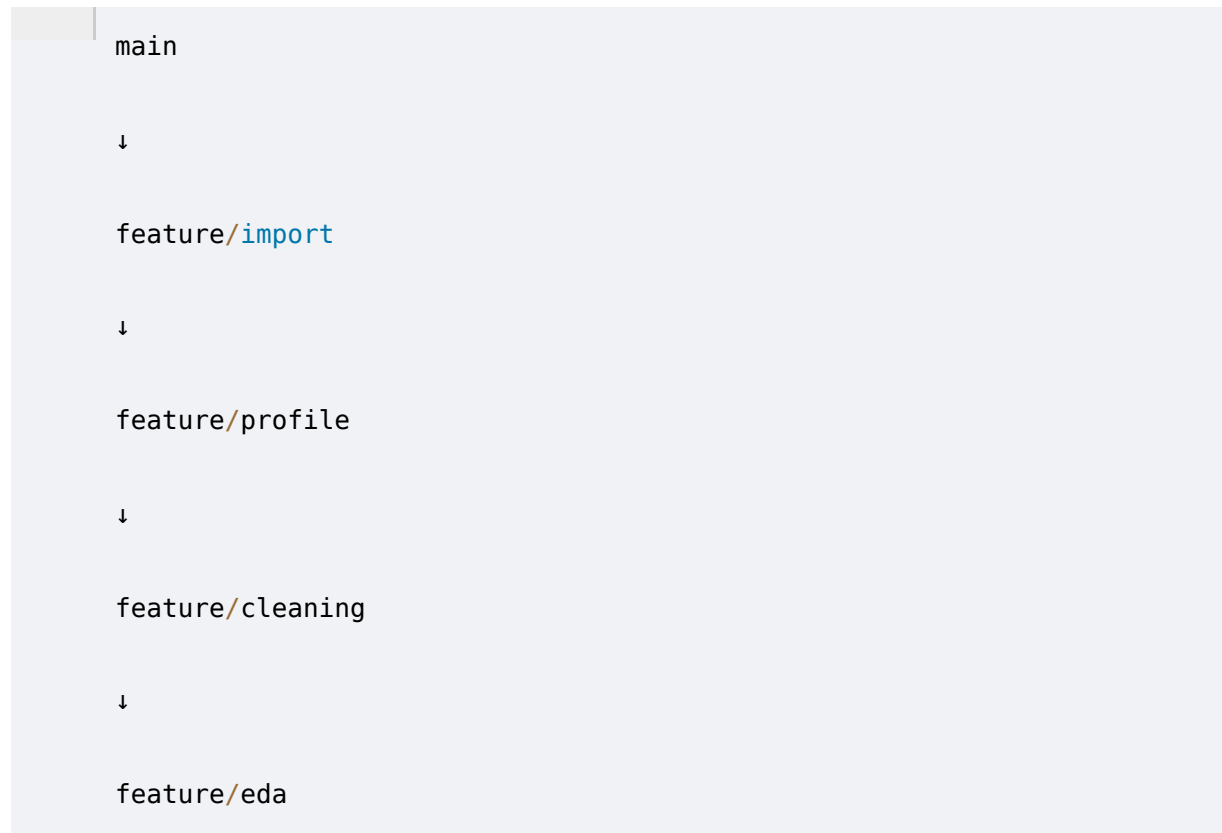
21. Development Workflow

For every feature:

1. Design
2. API
3. Backend
4. Tests
5. Frontend
6. Integration
7. Polish

Never skip testing.

22. Git Workflow



Merge only after testing.

23. AI Assistant Instructions

Whenever an AI assistant joins development, assume the following:

- Read this document first.
 - Do not redesign the architecture.
 - Respect existing folder structure.
 - Reuse existing services.
 - Never duplicate logic.
 - Prefer extension over modification.
 - Keep code production-ready.
-

24. Definition of Done

A feature is complete only if:

- Backend implemented.

- API documented.
 - Frontend integrated.
 - Tested.
 - Error handling included.
 - Loading state implemented.
 - Empty state implemented.
 - Responsive.
 - Matches design system.
-

25. Future Roadmap

Planned enhancements:

- Authentication
 - User Projects
 - Team Workspaces
 - Background Jobs
 - Explainable AI
 - Forecasting
 - Cloud Storage
 - Connectors (Google Sheets, SQL, etc.)
 - LLM-powered insights
-

26. Non-Goals (MVP)

The MVP intentionally excludes:

- Real-time collaboration
- User authentication
- Multi-tenancy
- Cloud storage
- Background job queues
- LLM integrations
- Time-series forecasting

This keeps the scope realistic for the current deadline.

27. AI Session Bootstrap Prompt

Every new AI session should begin with:

You are the Senior Principal Software Engineer **for** Prism **IQ**.

Read **MASTER_CONTEXT**.md before making any architectural or implementation decisions.

Do not change the project philosophy, architecture, database design, **API** conventions, **UI** design system, or folder structure unless explicitly instructed.

Maintain consistency **with** the documented standards.

28. Success Criteria

Prism IQ is considered successful when a user can:

- Import data from supported sources.
- Receive automatic profiling.
- Apply smart cleaning.
- Explore interactive visualizations.
- Train predictive models.
- Read actionable recommendations.
- Download a professional executive report.

All within a guided, intuitive workspace.

Final Recommendation (Very Important)

Veere, hun documentation almost **enterprise-level** ho chukki aa.

Par main ik hor cheez add karna chaunda haan jo honestly **99% developers nahi karde**.

 **ARCHITECTURE_DECISIONS.md** (ADR)

Eh file har major technical decision da reason preserve karegi.

Example:

- Why FastAPI instead of Django?
- Why PostgreSQL instead of MongoDB?
- Why local filesystem instead of storing CSVs in DB?
- Why Plotly + Recharts?

- Why pipeline-based navigation?
- Why repository pattern?
- Why synchronous MVP instead of Celery?

6 mahine baad jadon tussi project dubara kholo, tuhanu yaad rahega **sirf ki kita si nahi, balki kyun kita si.**

Eh document open-source projects te senior engineering teams bahut value karde ne.

Meri recommendation: `PROJECT_RULES.md` ton pehlan assi

`ARCHITECTURE_DECISIONS.md` banaiye. Is naal Prism IQ di documentation truly professional level te pahunch ju. 🚀